PLATFORM DOCUMENTATION

Workload Catalog

dealing cluster

CLUSTER	dealing cluster
AUDIENCE	Platform & on-call engineers
CLASSIFICATION	Internal — keep current as the platform evolves

Contents

> **Status as of generation:** the cluster has platform/infrastructure components only. The `production`, `staging`, and `development` namespaces are empty (no Deployments, StatefulSets, or DaemonSets). ArgoCD has zero registered Applications. This document captures the *platform* as-is and provides a template the team should fill in as application workloads land.

1. Namespace inventory

Namespace	Purpose	Environment	PSS enforce	Notes
`production`	App workloads (empty today)	production	restricted	Has 4 NetworkPolicies including `default-deny-all` — ready for apps
`staging`	App workloads (empty today)	staging	baseline	Same NetworkPolicy set as production
`development`	App workloads (empty today)	development	baseline	Only `allow-all-egress` + `default-deny-ingress`
`argocd`	GitOps controller	production-labeled	baseline	5 Deployments + 1 StatefulSet (argocd v2.13.0)
`monitoring`	Prometheus + KSM + Operator	platform	privileged	Required (Prometheus needs hostPath for scrape)
`cert-manager`	Certificate lifecycle	platform	restricted	3 Deployments (cert-manager v1.14.5)
`external-secrets`	Secrets sync controller	platform	(no label)	3 Deployments (v2.5.0) — **unconfigured** (no `ClusterSecretStore` defined)
`cilium-secrets`	Cilium-managed secrets	platform	(Helm-managed)	Hubble TLS, etc.
`kube-system`	Kubelet-managed core	system	(none)	RKE2 + Cilium + CoreDNS + kube-vip + Hubble UI
`kube-public`, `kube-node-lease`, `default`	System	system	(none)	Unused for workloads

Drift / quality flags to address

- **`external-secrets` namespace is missing the `pod-security.kubernetes.io/enforce` label.** Apply at least `baseline`.

- **No `ResourceQuota` or `LimitRange` in any namespace.** Recommended for `production` / `staging` / `development` so a runaway workload can't exhaust the cluster.
- **No PSS labels on `default`.** Leave or apply `restricted` (nothing should run there).

2. Platform components deployed

2.1 GitOps — ArgoCD `v2.13.0` (namespace: `argocd`)

Component	Replicas	Role
`argocd-server`	1	Web/API
`argocd-application-controller` (StatefulSet)	1	Reconciles Applications
`argocd-repo-server`	1	Pulls/renders Git repos
`argocd-applicationset-controller`	1	Multi-cluster app generation
`argocd-notifications-controller`	1	Notifications
`argocd-redis`	1	Cache (no persistence — restart loses cache, not state)

- Exposed: `Ingress argocd.dealing.internal` via Cilium IngressController (TLS via cluster-ca-issuer)
- **No Applications registered yet** → nothing is being GitOps-managed by ArgoCD right now. Workloads must either be Helm-installed (terraform), kubectl-applied, or registered as ArgoCD Applications going forward.

2.2 Certificates — cert-manager `v1.14.5` (namespace: `cert-manager`)

Component	Replicas
`cert-manager` (controller)	1
`cert-manager-cainjector`	1
`cert-manager-webhook`	1

ClusterIssuers configured:

Name	Type	Status	Use
`cluster-ca-issuer`	Internal CA	Ready	Default — used by `argocd.dealing.internal`, `hubble.cluster.internal`

Name	Type	Status	Use
`selfsigned-issuer`	Self-signed	Ready	Bootstrap / fallback
`letsencrypt-prod`	ACME public CA	NotReady	Configured but not currently functional
`letsencrypt-staging`	ACME public CA	NotReady	Configured but not currently functional

> **Action:** Decide whether Let's Encrypt is in scope. If yes, investigate why issuers are `NotReady` (DNS01/HTTP01 solver setup). If no, remove to reduce noise.

2.3 Secrets — external-secrets `v2.5.0` (namespace: `external-secrets`)

3 Deployments running, but **no `ClusterSecretStore` or `SecretStore` configured**, and **no `ExternalSecret` resources exist**. Effectively idle. To make functional: 1. Configure a backend (Vault / Azure KV / AWS SM / etc.) via `ClusterSecretStore` 2. Define `ExternalSecret` CRs to sync secrets

2.4 Observability — kube-prometheus-stack `80.4.1` (namespace: `monitoring`)

Component	Replicas	Resource state
`prometheus-kube-prometheus-stack-prometheus` (StatefulSet)	1	Single replica — no HA
`kube-prometheus-stack-operator`	1	
`kube-prometheus-stack-kube-state-metrics`	1	

- Remote-writes to **central Mimir** at `http://mimir.stackflow.org/api/v1/push` (X-Scope-OrgID: dealing)
- 17 `ServiceMonitor` resources scraping the cluster
- **No `Alertmanager` actively in use** (your alerting is Grafana → Teams; see §5)
- Prometheus retention / persistence: storage class TBD — verify

Companion: Grafana Alloy runs as **systemd on each VM** (not Kubernetes) — emits node metrics (`prometheus.exporter.unix`) and logs (`loki.source.file`) directly to Mimir/Loki. Not in the cluster inventory but part of the observability story.

2.5 Networking — Cilium `v1.18.3` (namespace: `kube-system`)

Component	Replicas	Role
`cilium` (DaemonSet)	6 (one per node)	CNI agent
`cilium-operator`	2 (HA)	Identity allocation, IPAM ops, garbage collection
`hubble-relay`	1	Flow aggregator
`hubble-ui`	1	Web UI for flow inspection
Envoy (embedded in `cilium-agent` pod, not a separate DaemonSet in this version)	n/a	L7 proxy used by the Cilium IngressController (handles every external HTTPS request hitting `10.10.120.140`). Also used for workload-level L7 visibility *when* a CiliumNetworkPolicy with `http: []` / `dns: []` rules is in effect — that part is off today, so workload pod-to-pod HTTP is NOT parsed for metrics.

Cluster mesh: `cluster.name=dealing, cluster.id=1` — single-cluster today. Mesh requires explicit ClusterMesh configuration if/when added.

LoadBalancer / Ingress: Cilium `IngressController` (default IngressClass), shared LB mode, single VIP `10.10.120.140` for all Ingress traffic. Gateway API CRDs installed but no HTTPRoutes defined.

2.6 Control-plane VIP — kube-vip (namespace: `kube-system`)

Component	Replicas	Role
`kube-vip-ds` (DaemonSet, host-network)	3 (one per master)	Manages the `10.10.120.138` control-plane VIP via leader election; failover when active master goes down

The masters' kubeconfig and worker join URLs all point at this VIP. If `kube-vip-ds` is unhealthy, new connections to the apiserver fail; existing TCP connections survive briefly.

2.7 RKE2-bundled add-ons (Helm-managed by RKE2 helm-controller, namespace: `kube-system`)

Component	Replicas	Role
`rke2-coredns-rke2-coredns` (Deployment)	2 (autoscaled)	Cluster DNS — Corefile customized to resolve `kubernetes` and `mimir.stackflow.org`
`rke2-coredns-rke2-coredns-autoscaler`	1	Scales CoreDNS replicas based on node count
`rke2-metrics-server`	1	Powers `kubectl top` and HPA resource metrics

Component	Replicas	Role
`rke2-snapshot-controller`	1	CSI volume snapshot controller (no usage today, no PVCs)

2.8 Control-plane static pods (one per master, host-network)

Defined as static pod manifests by RKE2; not visible in any Helm release. Restart only when their underlying file in `/var/lib/rancher/rke2/agent/pod-manifests/` changes (e.g., RKE2 upgrade, config change).

Static pod	Per master	Role
`kube-apiserver-dealing-m-N`	1	API server
`kube-controller-manager-dealing-m-N`	1	Reconciles core controllers (deployment, replicaset, endpoint, GC, …)
`kube-scheduler-dealing-m-N`	1	Pod scheduler
`etcd-dealing-m-N`	1	etcd cluster member
`cloud-controller-manager-dealing-m-N`	1	Currently a no-op (no cloud provider — bare-metal Proxmox). Could be disabled.

2.9 Storage

- **No PVCs, no PVs, no StorageClasses observed.**
- RKE2 normally ships a `local-path` provisioner — verify it's enabled and which default StorageClass exists if any.
- Worker nodes have a 20 GB data disk (`worker_data_disk_size_gb`) presumably intended for local-path PVs. With only 20 GB × 3 workers, total cluster persistent capacity is ~60 GB — tight if any stateful workload is planned.

2.10 Helm releases (the install source of truth)

Namespace	Release name	Status	Notes
`kube-system`	`rke2-cilium`	deployed	+ 1 superseded revision (today's nodeEncryption flip)
`kube-system`	`rke2-coredns`	deployed	
`kube-system`	`rke2-metrics-server`	deployed	

Namespace	Release name	Status	Notes
`kube-system`	`rke2-runtimeclasses`	deployed	RuntimeClass definitions (none used today)
`kube-system`	`rke2-snapshot-controller`	deployed	+ CRD release
`kube-system`	`rke2-snapshot-controller-crd`	deployed	
`argocd`	`argocd`	deployed	
`cert-manager`	`cert-manager`	deployed	
`external-secrets`	`external-secrets`	deployed	
`monitoring`	`kube-prometheus-stack`	deployed	+ 1 superseded revision (today's chart bump 58.2.2 → 80.4.1)

> **Action:** all releases except `argocd` and `external-secrets` are managed by Terraform (`terraform/observability/`, RKE2 helm-controller, etc.). `argocd` and `external-secrets` are also Terraform-managed but currently empty of *application* content. Confirm there's no Helm-installed-by-hand drift.

2.11 Cluster API surface — CRDs (65 total)

API group	Count	Owner
`generators.external-secrets.io`	17	external-secrets controller
`cilium.io`	13	Cilium agent / operator
`monitoring.coreos.com`	10	Prometheus Operator (ServiceMonitor, PodMonitor, PrometheusRule, Alertmanager, …)
`external-secrets.io`	6	external-secrets (ExternalSecret, ClusterSecretStore, …)
`cert-manager.io`	4	cert-manager (Certificate, Issuer, ClusterIssuer, CertificateRequest)
`argoproj.io`	3	ArgoCD (Application, AppProject, ApplicationSet)
`snapshot.storage.k8s.io`	3	RKE2-shipped CSI snapshot

API group	Count	Owner
`groupsnapshot.storage.k8s.io`	3	RKE2-shipped CSI group snapshot
`helm.cattle.io`	2	RKE2 helm-controller (HelmChart, HelmChartConfig)
`acme.cert-manager.io`	2	cert-manager ACME (Order, Challenge)
`k3s.cattle.io`	2	RKE2 add-on hooks

Practical note for the team: the most operationally relevant CRDs to learn are `Application` (ArgoCD), `ServiceMonitor` / `PrometheusRule` (Prometheus Operator), `CiliumNetworkPolicy` / `CiliumClusterwideNetworkPolicy` (Cilium), `Certificate` / `ClusterIssuer` (cert-manager), and `ExternalSecret` / `ClusterSecretStore` (external-secrets).

2.12 Admission webhooks (every API request passes through these)

Kind	Name	Owner	Failure mode
ValidatingWebhook	`cert-manager-webhook`	cert-manager	Certificate/Issuer create/update validation
MutatingWebhook	`cert-manager-webhook`	cert-manager	Defaults injection on Certificate objects
ValidatingWebhook	`externalsecret-validate`	external-secrets	ExternalSecret validation
ValidatingWebhook	`secretstore-validate`	external-secrets	(Cluster)SecretStore validation
ValidatingWebhook	`kube-prometheus-stack-admission`	Prometheus Operator	PrometheusRule / ServiceMonitor validation
MutatingWebhook	`kube-prometheus-stack-admission`	Prometheus Operator	PrometheusRule defaulting

Failure scenarios to understand: - If the `cert-manager-webhook` pod is down and a webhook timeout fires, you can't create/update `Certificate` or `Ingress` (with cert-manager annotations). - If `kube-prometheus-stack-admission` is down, `PrometheusRule` CRs become unmodifiable. - The webhooks' `failurePolicy` (Fail vs Ignore) determines whether requests are denied or allowed when the webhook is unavailable. Verify per webhook before deciding maintenance windows.

2.13 RBAC — who can do what

2.13.1 Subjects with `cluster-admin` (god-mode)

Subject	Kind	Source	Notes
`system:masters`	Group	Built-in (RBAC bootstrap)	Anyone whose client cert carries `O=system:masters` (admin kubeconfig).
`helm-rke2-cilium`	ServiceAccount (kube-system)	RKE2 helm-controller	Each `helm-` SA is a per-chart install identity; they need cluster-admin to install arbitrary Helm charts. The pods run only during install/upgrade Jobs, then exit. **The ClusterRoleBindings are permanent, though.**
`helm-rke2-coredns`	ServiceAccount (kube-system)	RKE2 helm-controller	Same as above
`helm-rke2-metrics-server`	ServiceAccount (kube-system)	RKE2 helm-controller	Same
`helm-rke2-runtimeclasses`	ServiceAccount (kube-system)	RKE2 helm-controller	Same
`helm-rke2-snapshot-controller`	ServiceAccount (kube-system)	RKE2 helm-controller	Same
`helm-rke2-snapshot-controller-crd`	ServiceAccount (kube-system)	RKE2 helm-controller	Same

> **Risk note:** any controller that can `create pods` in `kube-system` + `update sa/token` could mint a token for a helm-* SA and become cluster-admin. The kube-system PSS labels are absent — consider applying `restricted` or `baseline` if compatible.

2.13.2 Application ServiceAccounts (purpose-scoped, NOT cluster-admin)

Namespace	ServiceAccount	ClusterRoles bound	What they can do
`argocd`	`argocd-application-controller`	`argocd-application-controller`	Read/write across namespaces — Argo's reconcile loop

Namespace	ServiceAccount	ClusterRoles bound	What they can do
`argocd`	`argocd-server`	`argocd-server`	Read for the UI/API; impersonation for user-tied commands
`argocd`	`argocd-notifications-controller`	`argocd-notifications-controller`	Read Applications, push notifications
`cert-manager`	`cert-manager`	8 controller roles (challenges/ orders/issuers/ clusterissuers/…)	Issue/renew Certificates cluster-wide
`cert-manager`	`cert-manager-cainjector`	`cert-manager-cainjector`	Inject CA bundles into ValidatingWebhookConfiguration / APIService / etc.
`cert-manager`	`cert-manager-webhook`	`cert-manager-webhook:subjectaccessreviews`	SAR checks during admission
`external-secrets`	`external-secrets`	`external-secrets-controller`	Reconcile ExternalSecret → Secret (across all namespaces)
`external-secrets`	`external-secrets-cert-controller`	`external-secrets-cert-controller`	Manage its own webhook certs
`monitoring`	`kube-prometheus-stack-prometheus`	`kube-prometheus-stack-prometheus`	Read pods/services/nodes for service discovery
`monitoring`	`kube-prometheus-stack-operator`	`kube-prometheus-stack-operator`	Manage Prometheus, Alertmanager, ServiceMonitor CRs
`monitoring`	`kube-prometheus-stack-kube-state-metrics`	`kube-prometheus-stack-kube-state-metrics`	Read all KSM-covered resources cluster-wide

All of these are well-scoped — none of the application controllers carry cluster-admin. Good baseline.

2.13.3 Pre-handover RBAC actions

1. **Audit `system:masters` access** — who has admin kubeconfigs? Are they checked into a vault? Rotate cert expiry?
2. **Document the human-RBAC plan** — read-only viewer roles? Namespace-scoped admins? Currently there is no user RoleBinding visible (no humans defined as direct subjects).
3. **Decide on auditing** — `--audit-log-path` is on (`/var/lib/rancher/rke2/server/logs/audit.log`); the audit policy is at `rke2/configs/audit-policy.yaml` . Verify Alloy is shipping it to Loki for retention beyond local disk.

2.14 Resource requests / limits (per-workload)

> **Heads-up: this is the biggest quality issue I see in the cluster.** 16 of 22 workloads have **no CPU/memory requests OR limits**. Scheduler treats them as `BestEffort` QoS — first to be evicted under node pressure, and able to use unbounded CPU/memory.

Namespace	Workload	Kind	Rep	CPU req	CPU lim	Mem req	Mem lim	Notes
`argocd`	`argocd-server`	Deployment	1	100m	500m	256Mi	512Mi	✓
`argocd`	`argocd-application-controller`	StatefulSet	1	✗	✗	✗	✗	**The most resource-hungry Argo component — should have requests**
`argocd`	`argocd-applicationset-controller`	Deployment	1	✗	✗	✗	✗	
`argocd`	`argocd-notifications-controller`	Deployment	1	✗	✗	✗	✗	
`argocd`	`argocd-repo-server`	Deployment	1	✗	✗	✗	✗	git-clone workhorse — needs at least requests
`argocd`	`argocd-redis`	Deployment	1	✗	✗	✗	✗	
`cert-manager`	`cert-manager`	Deployment	1	✗	✗	✗	✗	
`cert-manager`	`cert-manager-cainjector`	Deployment	1	✗	✗	✗	✗	
`cert-manager`	`cert-manager-webhook`	Deployment	1	✗	✗	✗	✗	**Webhook in critical path — set requests so it doesn't get evicted**
`external-secrets`	`external-secrets`	Deployment	1	✗	✗	✗	✗	

Namespace	Workload	Kind	Rep	CPU req	CPU lim	Mem req	Mem lim	Notes
external-secrets	external-secrets-cert-controller	Deployment	1	✗	✗	✗	✗	
external-secrets	external-secrets-webhook	Deployment	1	✗	✗	✗	✗	Critical-path webhook
kube-system	cilium	DaemonSet	6	100m	2	512Mi	2Gi	✓
kube-system	cilium-operator	Deployment	2	✗	✗	✗	✗	
kube-system	hubble-relay	Deployment	1	100m	500m	64Mi	256Mi	✓
kube-system	hubble-ui	Deployment	1	✗	✗	✗	✗	
kube-system	kube-vip-ds	DaemonSet	3	✗	✗	✗	✗	**Control-plane VIP — should have requests**
kube-system	rke2-coredns-rke2-coredns	Deployment	2	100m	100m	128Mi	128Mi	✓
kube-system	rke2-coredns-rke2-coredns-autoscaler	Deployment	1	25m	100m	16Mi	64Mi	✓
kube-system	rke2-metrics-server	Deployment	1	100m	✗	200Mi	✗	requests only, no limits
kube-system	rke2-snapshot-controller	Deployment	1	✗	✗	✗	✗	
monitoring	prometheus-kube-prometheus-stack-prometheus	StatefulSet	1	✗	✗	✗	✗	**Prometheus has no limits — can OOM the node if cardinality explodes**
monitoring	kube-prometheus-stack-operator	Deployment	1	✗	✗	✗	✗	
monitoring	kube-prometheus-stack-kube-state-metrics	Deployment	1	✗	✗	✗	✗	

Suggested baseline requests (for a quick fix)

If you don't want to research each chart's optimal values, applying these "good-enough" baselines as Helm values is much better than nothing:

Workload class	CPU req	Mem req	CPU lim	Mem lim
Controllers (cert-manager, external-secrets, argocd-*)	50m	64Mi	500m	256Mi
Webhooks (cert-manager-webhook, external-secrets-webhook)	50m	64Mi	500m	128Mi
Prometheus (StatefulSet)	500m	2Gi	2	4Gi
kube-state-metrics	50m	64Mi	200m	256Mi
Prometheus operator	50m	100Mi	200m	200Mi
kube-vip-ds	50m	64Mi	100m	128Mi
hubble-ui	25m	64Mi	200m	256Mi
cilium-operator	100m	128Mi	1	1Gi

After applying, watch the actual usage for a week, then tune.

2.15 Container image inventory

Registry	Containers	Provider	Trust model
`docker.io` (Rancher mirrors `rancher/*`)	9	Rancher / SUSE	Mirrors of upstream Cilium / CoreDNS / KSM / snapshot-controller with Rancher's hardening + CVE backports. Image freshness depends on Rancher release cadence.
`quay.io`	11	Red Hat / various OSS projects (ArgoCD, cert-manager, Prometheus)	Upstream maintainers. Generally fastest path to upstream releases.
`ghcr.io`	4	OSS projects on GitHub (external-secrets, kube-vip)	Upstream maintainers via GitHub Container Registry.
`registry.k8s.io`	1	Kubernetes SIGs (kube-state-metrics)	First-party Kubernetes ecosystem.
`public.ecr.aws`	1	AWS public registry (redis)	Public mirror; same images as docker.io/redis.

Image-tag pinning posture

Posture	Count	Notes
Specific semver tag (`v1.18.3`, `v2.13.0`)	26	✓ Reproducible across pulls
Latest / floating tag	0	✓ None observed
Digest pin (`@sha256:…`)	0	Tag-only pins are mutable on the registry side; digest pins are immutable

Version inventory (cross-check for staleness/CVE)

Component	Running version	Notes
Cilium	v1.18.3	Current minor (1.18.x); 1.18.4 patches may be available
ArgoCD	v2.13.0	Current as of late 2024; 2.14 is the next
cert-manager	v1.14.5	Behind current (1.16+); review CHANGELOG for security fixes
external-secrets	v2.5.0	**Verify** — upstream external-secrets is at v0.x. This may be a fork or a wrong tag.
Prometheus	v3.8.0	Modern, on the v3 line
Prometheus Operator	v0.87.1	Reasonably current
kube-state-metrics	v2.17.0	Reasonably current
RKE2	v1.32.10+rke2r1	(from inventory.ini)
CoreDNS (Rancher hardened)	v1.13.1-build20251015	
kube-vip	v0.7.2	v0.8.x available; check changelog before upgrading

Pre-handover supply-chain actions

1. **Verify the external-secrets `v2.5.0` tag** — if it's a custom build, document the source; if it's wrong, replace with the upstream tag your team intends (e.g., `v0.10.x`).
2. **Decide on image-pull policy** — `IfNotPresent` (current default) is fine; consider an admission policy that blocks `:latest` for safety (Kyverno or similar).
3. **Schedule a CVE scan** — `trivy image` against the full inventory, repeat monthly.
4. **Consider digest pinning** for production-critical images (apiserver / cilium / etcd would be sensible candidates), if you want immutability against registry tampering.

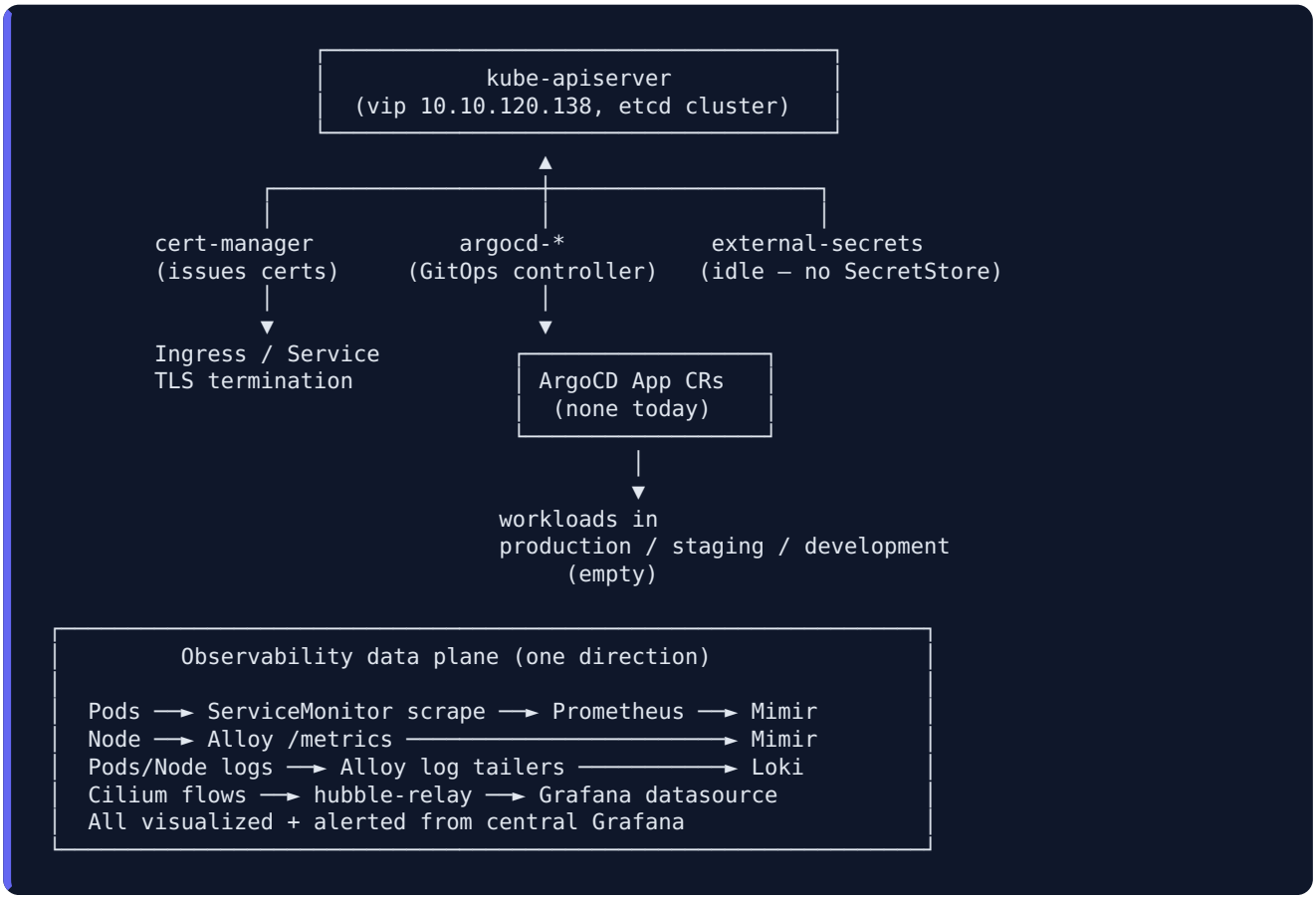
5. **Document image source policy** — which registries are allowed, and what your team's process is for adding new ones.

3. External exposure (what the outside world can reach)

Endpoint	Type	Backend	TLS issuer
`argocd.dealing.internal`	Ingress (cilium)	argocd/argocd-server	cluster-ca-issuer
`hubble.cluster.internal`	Ingress (cilium)	kube-system/hubble-ui	cluster-ca-issuer
`10.10.120.140:80/443`	LoadBalancer (Cilium L2)	shared IngressController	(per-ingress TLS)

> **Action:** Internal-only hostnames (`*.dealing.internal` , `cluster.internal`) require DNS resolution on the internal network. Verify your internal DNS resolver returns these.

4. Dependencies graph (platform → platform)



5. Alerting & escalation (per "Grafana → Teams" plan)

- **Alerts source:** Grafana managed alerts (NOT the bundled kube-prometheus-stack `PrometheusRule` resources — those would route through Alertmanager, which is not used here).
- **Notification target:** Microsoft Teams via webhook (Power Automate Workflows recommended; legacy "Incoming Webhook" being deprecated by Microsoft).
- **Severity tiers (proposed — fill in):**

Severity	Target	Examples
P1 (page)	Teams channel + (oncall human?)	etcd quorum lost, apiserver down, all nodes NotReady
P2 (notify)	Teams channel	One node NotReady, persistent OOM, etcd fsync p99 > 100ms, persistent volume <15%
P3 (digest)	Teams channel (daily)	High CPU throttling, Hubble drop rate elevated, alert fatigue items

> **Action:** team to decide who's on-call for each tier and document escalation in the runbook.

6. SLOs (template — humans must fill in)

Examples below are **placeholders** — actual numbers should reflect what your business needs.

Platform SLOs (apply to everyone using the cluster)

Service	SLI	SLO (proposed)	Failure budget / 30d
kube-apiserver availability	`sum(rate(apiserver_request_total{code=~"2..|3..|4.."})) / sum(rate(apiserver_request_total))`	99.9% over rolling 30d	43m 12s
kube-apiserver latency	`histogram_quantile(0.99, sum(rate(apiserver_request_duration_seconds_bucket{verb!~"WATCH"}[5m])) by (le))`	p99 < 1s	n/a (latency budget)
etcd availability	`etcd_server_has_leader == 1`	100%	(any drop = incident)
etcd write latency	`histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m]))`	p99 < 100 ms	(above is "fast"; >100 ms = degraded)

Service	SLI	SLO (proposed)	Failure budget / 30d
Pod scheduling latency	`histogram_quantile(0.99, sum(rate(scheduler_pod_scheduling_duration_seconds_bucket[5m])) by (le))`	p99 < 5s	n/a
Node availability	`count(kube_node_status_condition{condition="Ready",status="true"}) / count(kube_node_info)`	100% of 6 nodes	n/a (any drop = page)

Application SLOs

> Once apps land in `production`, each app/team should define: - **SLI definition** (request success rate, latency p99, custom metric) - **SLO target** (e.g., 99.5% success, p99 < 200ms) - **Error budget** (= 1 − SLO, allocated over the rolling window) - **Burn rate alerts** (2 × budget burn over 1h → notify; 14.4 × over 1h → page)

A useful template is the Google SRE "multi-window, multi-burn-rate" pattern: https://sre.google/workbook/alerting-on-slos/

7. Backup / restore status

Item	Status
etcd local snapshots	Configured (`*/6h cron`, retention 10), stored on `scsi1` etcd disk on each master
etcd external/offsite snapshots	**Unverified** — config comment claims it exists; no evidence found. **Action:** verify and document the offsite destination.
Persistent volumes	None today
Workload secrets backup	Tied to external-secrets backend (currently unconfigured)
Restore drill performed	No — required before handover

8. Capacity snapshot (today)

Resource	Current	Capacity
Nodes	6 (3 master + 3 worker)	n/a

Resource	Current	Capacity
Total CPU	~1.97% used	6 × vCPUs (need exact count — `kube_node_status_capacity` after Alloy fix)
Total memory	TBD	TBD
Total pod slots	49 active	6 × 110 max-pods = 660
etcd DB size	small (TBD)	8 GB quota
Local PV	0 used	~60 GB across workers

9. Open gaps to address before handover

1. **No `ExternalSecret` / `ClusterSecretStore`** — external-secrets is idle. Either configure or remove.
2. **`letsencrypt-*` ClusterIssuers `NotReady`** — fix or remove.
3. **No `ResourceQuota` / `LimitRange` in any namespace** — apply for `production`, `staging`, `development`.
4. **PSS label missing on `external-secrets` namespace** — add at least `baseline`.
5. **Many platform pods have no CPU/memory requests or limits** (cert-manager, external-secrets, monitoring stack) — this lets the scheduler over-pack nodes. Set realistic requests on every Deployment.
6. **No ArgoCD Applications** — define how workloads get deployed (ArgoCD Applications? Helm directly? Terraform? — pick one and document).
7. **etcd offsite backups unverified.**
8. **No restore drill performed.**
9. **No Alertmanager → Teams configured AND no Grafana managed alerts defined yet.** Pick one path and implement.

10. Maintenance windows / change control

> Team to fill in: - Maintenance window cadence (weekly? monthly?) - Who approves prod changes? - How are CRD / Helm chart upgrades tested before prod? - Rollback procedure for a failed RKE2 / Cilium upgrade?

11. Useful one-liner inventory commands

```
# Workload-by-namespace summary
kubectl get deploy,sts,ds -A

# Currently-running pods per namespace
kubectl get pods -A | awk '{print $1}' | sort | uniq -c

# What's externally exposed
kubectl get ingress,svc -A | grep -E 'LoadBalancer|Ingress'

# ArgoCD app list (when populated)
kubectl -n argocd get applications

# Cluster health one-shot
kubectl get nodes -o wide
kubectl get --raw '/readyz?verbose' | grep -v ok
kubectl -n kube-system get pods -l 'tier=control-plane'
```

Generated: see commit log for date. Update this doc whenever a new app lands, a namespace is added, or the alerting path changes.